



PONTIFICIA
UNIVERSIDAD
CATÓLICA
DE CHILE



PONTIFICIA
UNIVERSIDAD
CATÓLICA
DE CHILE

Fundamentos Matemáticos de Inteligencia Artificial

IMT3850. Clase 6

Manuel A. Sánchez
2026.04.20

Probabilidades. Contenidos:

- 1 Teorema de Bayes y Naïve Bayes
- 2 Variables Aleatorias y Distribuciones
- 3 Valor esperado y Varianza
- 4 Variables Aleatorias Multivariadas
- 5 Aplicación: Algoritmos Aleatorizados
- 6 La Maldición de la Dimensionalidad
- 7 Desigualdades de Concentración
- 8 Muestras
- 9 Leyes Límite (LLN y CLT)

Teorema de Bayes y Naïve Bayes

Teorema de Bayes (Repaso)

Theorem (Ley de Bayes)

Asuma que $E_1, E_2, \dots, E_n$ son eventos mutuamente disjuntos tales que $\bigcup_{j=1}^n E_j = \Omega$.
Entonces, para un evento observado B :

$$P(E_j|B) = \frac{P(B|E_j)P(E_j)}{\sum_{i=1}^n P(B|E_i)P(E_i)}$$

Vocabulario en Ciencia de Datos:

$P(E_j)$: **Prior** (Nuestra creencia inicial antes de ver los datos).

$P(B|E_j)$: **Likelihood** (Verosimilitud de observar B si E_j fuera cierto).

$P(E_j|B)$: **Posterior** (Nuestra creencia actualizada tras ver la evidencia B).

Teorema de Bayes- drug testing

Test es 90% de precisión. Tenemos un 10% de uso de la droga.

Eventos. A : score del test. B : si usa la droga.

La probabilidad de que el test sea positivo dado que usa la droga es: $P(A|B) = 0.9$
(sensitivity)

La probabilidad de que el test sea negativo dado que no usa la droga es:
 $P(A^c|B^c) = 0.9$ (specificity)

La probabilidad de que use la droga dado que el test es positivo es:

$$P(B|A) = \frac{P(A|B)P(B)}{P(A|B)P(B) + P(A|B^c)P(B^c)} = \frac{(0.9)(0.1)}{(0.9)(0.1) + (0.1)(0.9)} = 0.5$$

Asuma ahora que la specificity es 0.8. Entonces

$$P(B|A) = \frac{P(A|B)P(B)}{P(A|B)P(B) + P(A|B^c)P(B^c)} = \frac{(0.9)(0.1)}{(0.9)(0.1) + (0.2)(0.9)} = 1/3$$

Introducción al Clasificador Naïve Bayes

El **Clasificador Naïve Bayes** es un modelo probabilístico fundamental en Inteligencia Artificial, basado en la aplicación directa del Teorema de Bayes.

El Problema de Clasificación: Dado un vector de atributos $x = [x_1, x_2, \dots, x_n]^T \in \mathbb{R}^n$, queremos asignar x a una de K clases posibles $\{C_1, C_2, \dots, C_K\}$.

Utilizando el Teorema de Bayes, calculamos la **probabilidad a posteriori** de cada clase dado el dato x :

$$P(C_k|x) = \frac{P(x|C_k)P(C_k)}{P(x)}$$

Nuestra regla de decisión será elegir la clase C_k que maximice esta probabilidad.

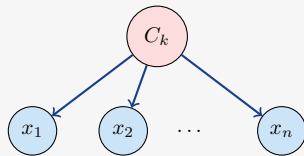
La Hipótesis "Naïve" (Ingenua)

Calcular la probabilidad conjunta $P(x|C_k) = P(x_1, \dots, x_n|C_k)$ es computacionalmente intratable en altas dimensiones debido a la **maldición de la dimensionalidad** (requeriría observar todas las combinaciones posibles de atributos).

El modelo asume "ingenuamente" que todos los atributos x_i son **mutuamente independientes** dada la clase C_k .

$$P(x_1, \dots, x_n|C_k) = P(x_1|C_k) \cdot P(x_2|C_k) \cdots P(x_n|C_k) = \prod_{i=1}^n P(x_i|C_k)$$

A pesar de que esta hipótesis casi nunca se cumple en el mundo real (ej. en un texto, la palabra "oferta" y "descuento" sí están correlacionadas), el modelo funciona sorprendentemente bien en la práctica.



La Regla de Decisión (MAP)

Sustituyendo el supuesto ingenuo en el Teorema de Bayes, obtenemos:

$$P(C_k|x) = \frac{P(C_k) \prod_{i=1}^n P(x_i|C_k)}{P(x)}$$

Optimización: Dado que el denominador $P(x)$ (la evidencia) es constante y exactamente igual para todas las clases, podemos ignorarlo para propósitos de maximización. Así, obtenemos la regla de decisión **Maximum A Posteriori (MAP)**:

$$\hat{y} = \arg \max_{k \in \{1, \dots, K\}} \left(P(C_k) \prod_{i=1}^n P(x_i|C_k) \right)$$

$P(C_k)$: **Prior** (Probabilidad a priori de la clase).

$\prod P(x_i|C_k)$: **Likelihood** (Verosimilitud de los datos).

Inestabilidad Numérica y el Truco del Logaritmo

Problema Computacional (Underflow): Multiplicar decenas de probabilidades (números entre 0 y 1) como $\prod_{i=1}^n P(x_i|C_k)$ genera un número tan pequeño que la precisión de punto flotante del computador lo redondeará a exactamente 0.

Solución (Log-Probabilidades): Aplicamos el logaritmo natural, el cual es una función monótona creciente (no altera cuál es la clase máxima) y transforma las multiplicaciones en sumas:

$$\log \left(P(C_k) \prod_{i=1}^n P(x_i|C_k) \right) = \log P(C_k) + \sum_{i=1}^n \log P(x_i|C_k)$$

Regla MAP en la práctica:

$$\hat{y} = \arg \max_k \left(\log P(C_k) + \sum_{i=1}^n \log P(x_i|C_k) \right)$$

Entrenamiento del Modelo (Fase de Ajuste)

"Entrenar" un modelo Naïve Bayes consiste simplemente en usar estadística descriptiva para calcular los parámetros a partir del conjunto de entrenamiento de tamaño N :

Cálculo del Prior $P(C_k)$: Frecuencia relativa de la clase.

$$P(C_k) = \frac{N_{C_k}}{N}$$

Cálculo de la Verosimilitud $P(x_i|C_k)$: Depende del tipo de datos.

Datos Categóricos (Multinomial/Bernoulli): Conteo de frecuencias de la palabra o atributo i en la clase k .

Datos Continuos (Gaussiano): Se asume que $P(x_i|C_k) \sim \mathcal{N}(\mu_{ik}, \sigma_{ik}^2)$. El entrenamiento consiste en calcular la media y la varianza empírica del atributo i para la clase k .

El Dataset de Vinos (Wine Dataset)

Descripción del Problema:

Contamos con los resultados de un análisis químico realizado a vinos cultivados en la misma región de Italia, pero derivados de **tres cultivos diferentes** (Clases 0, 1 y 2).

El análisis determinó las cantidades de **13 componentes** presentes en cada uno de los tres tipos de vinos.

Objetivo: Entrenar un modelo de Inteligencia Artificial capaz de adivinar el tipo de vino (la clase) basándose únicamente en sus 13 mediciones químicas.

Selección del Modelo: Naïve Bayes Gaussiano

A diferencia del filtro de Spam (donde contábamos palabras discretas), nuestros datos ahora son números **continuos** (ej. Alcohol = 13.2%).

La Asunción Gaussiana: Para modelar la verosimilitud (Likelihood) $P(x_i|C_k)$, el clasificador asume que los valores continuos asociados a cada clase están distribuidos según una **Distribución Normal (Gaussiana)**.

$$P(x_i|C_k) = \frac{1}{\sqrt{2\pi\sigma_{ik}^2}} \exp\left(-\frac{(x_i - \mu_{ik})^2}{2\sigma_{ik}^2}\right)$$

Entrenamiento: El algoritmo simplemente agrupa los vinos por su clase y calcula la **media** (μ) y la **varianza** (σ^2) de cada uno de los 13 atributos químicos.

Implementación Práctica: Scikit-Learn

Entrenar este modelo toma solo un par de líneas utilizando la librería `scikit-learn`.

Clasificación en 5 líneas

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

X, y = load_wine(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

modelo = GaussianNB()          # 1. Instanciar asumiendo dist. Normal
modelo.fit(X_train, y_train)    # 2. Entrenar: Priors, Medias y Varianzas
y_pred = modelo.predict(X_test) # 3. Predecir: Bayes a nuevos datos

print(f"Precisión: {accuracy_score(y_test, y_pred) * 100}%")
```

Implementación en Numpy desde cero

Para entender qué hace internamente `modelo.fit()`, basta con usar:

Calculando los Parámetros (Training)

```
import numpy as np

# Para cada clase (0, 1, 2), calculamos sus estadísticos
medias = []
varianzas = []
priors = []
for c in np.unique(y_train):
    X_c = X_train[y_train == c] # Aislamos los vinos de la clase 'c'
    medias.append(X_c.mean(axis=0)) # Vector de medias (13 features)
    varianzas.append(X_c.var(axis=0)) # Vector de varianzas (13 features)
    priors.append(X_c.shape[0] / float(len(y_train))) # P(C_k)
```

En la fase predictiva, simplemente evaluamos la densidad Gaussiana con estas medias y varianzas y sumamos los logaritmos

Variables Aleatorias y Distribuciones

Variables Aleatorias

En el mundo real observamos "eventos" (ej. un usuario hace clic, un sensor mide temperatura). Para que un computador procese esto, necesitamos mapearlo a números.

Definición Formal: Una **variable aleatoria** X es una función medible $X : \Omega \rightarrow \mathbb{R}$ que asigna un valor numérico a cada resultado de un experimento, de modo que podemos medir su probabilidad.

V.A. Discreta: Toma valores en un conjunto finito o numerable.

Uso en ML: Etiquetas de Clasificación (Perro=0, Gato=1), Conteo de palabras.

V.A. Continua: Toma cualquier valor en un intervalo real $\mathbb{R}$.

Uso en ML: Objetivos de Regresión (Precio de una casa), Pesos de una Red Neuronal.

Función de Distribución de Probabilidad

Función de Distribución Acumulada (CDF): Es la herramienta fundamental para caracterizar una V.A. X , denotada por $F(x)$:

$$F(x) = P(X \leq x) = P(\{\omega \in \Omega : X(\omega) \leq x\})$$

Es una función monótona no decreciente.

$$\lim_{x \rightarrow -\infty} F(x) = 0 \quad \text{y} \quad \lim_{x \rightarrow \infty} F(x) = 1$$

Variables Aleatorias Discretas: Son aquellas cuyo rango es un conjunto finito o **infinito numerable**. La probabilidad de un valor puntual se define mediante la suma de las probabilidades de los resultados que mapean a ese valor:

$$P(X = x) = \sum_{\omega \in \Omega: X(\omega)=x} P(\omega)$$

Función de Masa de Probabilidad

Para una V.A. discreta, definimos la **Función de Masa de Probabilidad** (función de densidad) $f(x)$ como:

$$f(x) = P(X = x)$$

donde se satisfacen las propiedades: $0 \leq f(x) \leq 1$ y $\sum_x f(x) = 1$.

Propiedad de la CDF Discreta: La función $F(x)$ es una **función constante a trozos** (monótona no decreciente) que presenta saltos en los puntos donde existe masa de probabilidad:

$$F(x) = \sum_{x_i \leq x} f(x_i)$$

El tamaño del salto en x_i es exactamente $f(x_i)$.

Ejemplo 1: Variable Aleatoria (Caso Equiprobable)

Consideremos el lanzamiento de dos monedas honestas. Definimos la variable aleatoria X como el **número total de caras**.

El espacio muestral es $\Omega = \{cc, cs, sc, ss\}$. La función $X : \Omega \rightarrow \mathbb{R}$ realiza el siguiente mapeo:

$$X(cc) = 2, \quad X(cs) = 1, \quad X(sc) = 1, \quad X(ss) = 0$$

Los valores posibles que toma X son $\{0, 1, 2\}$. Calculamos su **Función de Masa de Probabilidad**:

$$P(X = 2) = P(\{cc\}) = 1/4$$

$$P(X = 1) = P(\{cs, sc\}) = 2/4 = 1/2$$

$$P(X = 0) = P(\{ss\}) = 1/4$$

Ejemplo 2: Variable Aleatoria (Caso No Equiprobable)

Tenemos una urna con bolas verdes (v) y rojas (r). La probabilidad de extraer una bola verde es 0.3.

Experimento: Extraemos dos bolas **con reemplazo** (eventos independientes). Definimos X como el **número de bolas verdes extraídas**. Valores posibles: $\{0, 1, 2\}$.

Calculamos la probabilidad para cada valor de X :

$$P(X = 2) = P(\{v, v\}) = P(v) \cdot P(v) = (0.3)(0.3) = \mathbf{0.09}$$

$$P(X = 1) = P(\{v, r\}) + P(\{r, v\}) = (0.3)(0.7) + (0.7)(0.3) = \mathbf{0.42}$$

$$P(X = 0) = P(\{r, r\}) = (0.7)(0.7) = \mathbf{0.49}$$

Comprobación de sanidad: $0.09 + 0.42 + 0.49 = 1.0$ (La suma de todas las masas de probabilidad debe ser 1).

Modelos de Probabilidad Discreta

$P(X = x)$ (Función de Masa de Probabilidad)

Nombre	Notación	Función de Masa	Parámetros
Bernoulli	$\text{Bern}(p)$	$p^x(1-p)^{1-x}$	$p \in [0, 1]$
Binomial	$\text{Bin}(n, p)$	$\binom{n}{x} p^x (1-p)^{n-x}$	$n \in \mathbb{N}, p \in [0, 1]$
Multinomial	$\text{Mult}(n, \mathbf{p})$	$\frac{n!}{x_1! \cdots x_k!} p_1^{x_1} \cdots p_k^{x_k}$	$\sum p_i = 1, \sum x_i = n$
Poisson	$\text{Pois}(\lambda)$	$\frac{e^{-\lambda} \lambda^x}{x!}$	$\lambda > 0$
Geométrica	$\text{Geom}(p)$	$(1-p)^{x-1} p$	$p \in (0, 1]$
Hipergeom.	$\text{Hyper}(N, K, n)$	$\frac{\binom{K}{x} \binom{N-K}{n-x}}{\binom{N}{n}}$	$N, K, n \in \mathbb{N}$

Ejemplo: Control de Calidad (Dist. Hipergeométrica)

Escenario: Muestreo sin reemplazo de un lote finito.

N : Número total de items en el lote.

K : Número de items defectuosos en el lote.

n : Tamaño de la muestra extraída.

x : Número de items defectuosos encontrados en la muestra.

La probabilidad de encontrar exactamente x defectuosos se calcula como:

$$f(x) = P(X = x) = \frac{\binom{K}{x} \binom{N-K}{n-x}}{\binom{N}{n}}$$

Interpretación en Ciencia de Datos: Si observamos un valor x muy alto, esto nos da evidencia estadística para rechazar el lote o inferir que el parámetro K es mayor de lo reportado (Inferencia).

Variables Aleatorias Continuas y Densidad (PDF)

A diferencia del caso discreto, una **variable aleatoria continua** puede tomar cualquier valor dentro de un intervalo de números reales (ej. tiempo de respuesta de un servidor, o el valor del peso en una red neuronal).

Función de Densidad de Probabilidad (PDF): Una V.A. continua X se caracteriza por una función $f(x) \geq 0$ tal que el área total bajo la curva es exactamente 1 ($\int_{-\infty}^{\infty} f(x)dx = 1$).

En el caso continuo, la probabilidad de observar un valor **exacto** es siempre cero: $P(X = c) = 0$. La densidad $f(x)$ **NO** es una probabilidad; solo podemos calcular probabilidades sobre **intervalos**.

Probabilidades Continuas

Función de Distribución Acumulada (CDF): Para una V.A. continua, la CDF (probabilidad acumulada) se obtiene integrando la función de densidad desde $-\infty$ hasta el valor x :

$$F(x) = P(X \leq x) = \int_{-\infty}^x f(u)du$$

Probabilidad en un Intervalo: La probabilidad de que la variable caiga entre dos valores a y b corresponde geoméricamente al **área bajo la curva** de la PDF:

$$P(a < X \leq b) = F(b) - F(a) = \int_a^b f(x)dx$$

Nota en IA: Calcular probabilidades continuas se reduce a un problema de integración (o más frecuentemente, aproximación numérica).

Modelos de Probabilidad Continua

Nombre	Notación	Función de Densidad $f(x)$	Dominio
Uniforme	$\text{Unif}(a, b)$	$\frac{1}{b-a}$	$x \in [a, b]$
Normal	$\mathcal{N}(\mu, \sigma^2)$	$\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$	$x \in \mathbb{R}$
Exponencial	$\text{Exp}(\lambda)$	$\lambda e^{-\lambda x}$	$x \geq 0$
Gamma	$\text{Gamma}(\alpha, \beta)$	$\frac{\beta^\alpha}{\Gamma(\alpha)} x^{\alpha-1} e^{-\beta x}$	$x > 0$
Beta	$\text{Beta}(\alpha, \beta)$	$\frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)} x^{\alpha-1} (1-x)^{\beta-1}$	$x \in [0, 1]$
Student-t	t_ν	$\frac{\Gamma(\frac{\nu+1}{2})}{\sqrt{\nu\pi}\Gamma(\frac{\nu}{2})} \left(1 + \frac{x^2}{\nu}\right)^{-\frac{\nu+1}{2}}$	$x \in \mathbb{R}$

Nota en IA: La distribución Normal fundamenta el Error Cuadrático Medio (MSE). Beta y Gamma se utilizan como distribuciones a priori en inferencia Bayesiana.

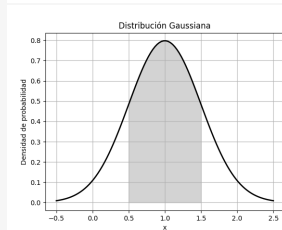
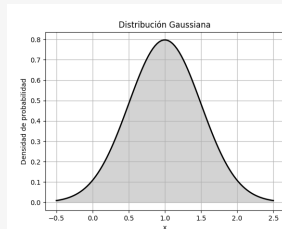
Gaussiana

1d. La función de densidad de una distribución Normal (Gaussiana) con media μ y varianza σ^2 es

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

Sabemos que:

$$\int_{-\infty}^{\infty} f(x) dx = 1$$
$$\int_{\mu-\sigma}^{\mu+\sigma} f(x) dx = \frac{1}{\sqrt{2\pi}} \int_{-1}^1 \exp\left(-\frac{x^2}{2}\right) dx \approx \frac{2}{3}$$



Aproximaciones de la Distribución Binomial

Si $X \sim \text{Bin}(n, p)$, calcular los factoriales para n grande es intratable. Tenemos dos grandes aproximaciones asintóticas:

1. Aproximación de Poisson (Eventos Raros): Si $n \rightarrow \infty$ y $p \rightarrow 0$ tal que $\lambda = np$ se mantiene constante.

$$P(X = k) \approx \frac{\lambda^k}{k!} \exp(-\lambda)$$

2. Aproximación Normal (De Moivre-Laplace): Si n es grande y p está balanceado (usualmente $np > 5$ y $n(1 - p) > 5$).

$$X \approx \mathcal{N}(\mu = np, \sigma^2 = np(1 - p))$$

Peligro Predictivo: Si intentamos aproximar una Binomial de un evento muy raro ($n = 1000, p = 1/1000$) usando la Normal, obtendríamos $\mathcal{N}(1, \approx 1)$. Esto implicaría un 16% de probabilidad de obtener valores negativos. Esto no tiene sentido físico para un conteo. Aquí la Poisson es la herramienta estadística correcta.

Poisson

La probabilidad que un foco de luz se apague es $p = 0.002$ y tenemos $n = 10000$ de estos focos en la carretera, entonces

$$P(\text{exactamente } k \text{ luces se apaguen}) \sim \text{Poisson}(\lambda), \quad \lambda = np = 20.$$

El uso práctico de los Modelos Continuos en IA

Más allá de la distribución Normal (que veremos en detalle con el Teorema del Límite Central), dos distribuciones continuas son vitales en la simulación estocástica:

1. Distribución Uniforme $\text{Unif}(0, 1)$: Es el motor de cualquier simulación en computador. Funciones como `np.random.rand()` generan números bajo esta distribución. Mediante el *Método de Transformada Inversa*, una $\text{Unif}(0, 1)$ puede transformarse en **cualquier** otra distribución continua.

2. Distribución Exponencial $\text{Exp}(\lambda)$: Modela el tiempo que transcurre entre eventos en un proceso de Poisson continuo. Se caracteriza por la propiedad de "Pérdida de Memoria" (*Memoryless*).

$$P(X > t + s | X > s) = P(X > t)$$

Aplicación: Tiempo de espera en colas de servidores web, tiempo hasta el próximo fallo de un clúster de GPUs.

Ejemplo 3: Tiempo de Falla en un Clúster de IA

Escenario: El tiempo T (en horas) hasta que una tarjeta gráfica (GPU) falla durante el entrenamiento de un modelo de Deep Learning sigue una distribución Exponencial con tasa $\lambda = 0.01$ fallos/hora.

La función de densidad es $f(t) = 0.01e^{-0.01t}$ para $t \geq 0$.

Pregunta A: ¿Cuál es la probabilidad de que la GPU sobreviva *más* de 100 horas de entrenamiento ininterrumpido?

Solución: Integramos la densidad desde 100 hasta ∞ :

$$\begin{aligned} P(T > 100) &= \int_{100}^{\infty} 0.01e^{-0.01t} dt \\ &= \left[-e^{-0.01t} \right]_{100}^{\infty} = (0) - (-e^{-0.01 \times 100}) \\ &= e^{-1} \approx \mathbf{0.3678} \quad (36.8\%) \end{aligned}$$

Ejemplo 4: La Pérdida de Memoria en Acción

Pregunta B: Supongamos que la GPU ya lleva 50 horas entrenando sin fallar. ¿Cuál es la probabilidad de que sobreviva *otras* 100 horas más (es decir, llegar a 150 horas)?

Solución: Usamos la fórmula de probabilidad condicional $P(A|B) = \frac{P(A \cap B)}{P(B)}$:

$$\begin{aligned} P(T > 150 \mid T > 50) &= \frac{P(T > 150 \text{ y } T > 50)}{P(T > 50)} \\ &= \frac{P(T > 150)}{P(T > 50)} = \frac{e^{-0.01 \times 150}}{e^{-0.01 \times 50}} \\ &= e^{-0.01(150-50)} = e^{-0.01 \times 100} \\ &= e^{-1} \approx \mathbf{0.3678} \quad (36.8\%) \end{aligned}$$

La probabilidad es exactamente la misma que si la GPU fuera nueva. El componente "no recuerda" que ya ha trabajado 50 horas.

Clasificación Binaria y la Distribución de Bernoulli

En Inteligencia Artificial, el problema más común es la **clasificación binaria** (ej. predecir si un tumor es maligno o benigno, si un correo es spam o no).

Codificación de Etiquetas: Definimos la respuesta correcta como $y \in \{0, 1\}$.

El Modelo Predictivo: Nuestra red neuronal (o modelo logístico) procesa un dato x y retorna una probabilidad estimada $\hat{p} \in (0, 1)$ de que la clase sea 1.

Si asumimos que cada etiqueta y es una variable aleatoria generada por un proceso de Bernoulli con parámetro $\hat{p}$, su Función de Masa de Probabilidad se escribe compactamente como:

$$P(Y = y|x) = \hat{p}^y(1 - \hat{p})^{1-y}$$

Comprobación: Si $y = 1$, la fórmula retorna $\hat{p}$. Si $y = 0$, la fórmula retorna $(1 - \hat{p})$.

Optimización: Máxima Verosimilitud (MLE)

Supongamos que tenemos un conjunto de entrenamiento con N datos independientes: $\{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$. Nuestro modelo produce N predicciones $\{\hat{p}_1, \hat{p}_2, \dots, \hat{p}_N\}$.

La Función de Verosimilitud (Likelihood): ¿Cuál es la probabilidad de que nuestro modelo haya generado exactamente las etiquetas y que observamos en la realidad? Multiplicamos las probabilidades individuales:

$$\mathcal{L}(\hat{p}) = \prod_{i=1}^N P(Y = y_i | x_i) = \prod_{i=1}^N \hat{p}_i^{y_i} (1 - \hat{p}_i)^{1-y_i}$$

Objetivo del Machine Learning: Queremos ajustar los pesos de nuestra red neuronal para **maximizar** esta verosimilitud $\mathcal{L}(\hat{p})$.

El Nacimiento de la "Cross-Entropy Loss"

Maximizar un producto de miles de probabilidades causa *underflow* numérico (las multiplicaciones colapsan a 0).

Log-Verosimilitud Negativa: 1. Aplicamos el logaritmo natural ($\log$) para transformar productos en sumas.
2. Multiplicamos por -1 porque en optimización computacional (PyTorch/TensorFlow) los algoritmos siempre buscan **minimizar** una función de pérdida (Loss Function).

$$\begin{aligned}\text{Loss} &= -\log \mathcal{L}(\hat{p}) = -\log \left(\prod_{i=1}^N \hat{p}_i^{y_i} (1 - \hat{p}_i)^{1-y_i} \right) \\ &= -\sum_{i=1}^N [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)]\end{aligned}$$

Valor esperado y Varianza

Valor esperado

Definición. El valor esperado de una variable aleatoria continua X con función de densidad f , denotada por $E[X]$ es definido por

$$E[X] = \int_{-\infty}^{\infty} x f(x) dx.$$

Definición. El valor esperado de una variable aleatoria discreta X , denotado por $E[X]$, está dado por

$$E[X] = \sum_x x f(x) = \sum_i i P(X = i)$$

donde la suma es sobre todos los valores en el rango de X .

Propiedades del Valor Esperado

- Si aplicamos una función $h(X)$ a nuestra variable aleatoria, su valor esperado es el promedio ponderado de esa función:

$$E[h(X)] = \int_{-\infty}^{\infty} h(x)f(x)dx \quad (\text{Caso Continuo})$$

$$E[h(X)] = \sum_x h(x)f(x) \quad (\text{Caso Discreto})$$

- **Linealidad de la Esperanza** : Sean X e Y variables aleatorias, y a, b, c constantes reales. Entonces:

$$E[X + Y] = E[X] + E[Y]$$

$$E[aX + b] = aE[X] + b$$

$$\implies E\left[\sum_{i=1}^n c_i X_i\right] = \sum_{i=1}^n c_i E[X_i]$$

Ejemplo 1: Suma de Dos Dados

Sea X la variable aleatoria que representa la suma de los resultados al lanzar 2 dados honestos.

Método 1: Fuerza Bruta (Sumando la PMF)

$$\begin{aligned} E[X] &= \sum_{i=2}^{12} iP(X = i) \\ &= 2 \left(\frac{1}{36} \right) + 3 \left(\frac{2}{36} \right) + \cdots + 7 \left(\frac{6}{36} \right) + \cdots + 12 \left(\frac{1}{36} \right) = \mathbf{7} \end{aligned}$$

Método 2: Usando Linealidad (El atajo) Definimos X_1 como el resultado del primer dado y X_2 como el segundo. Sabemos trivialmente que $E[X_1] = E[X_2] = 3.5$.

$$E[X] = E[X_1 + X_2] = E[X_1] + E[X_2] = 3.5 + 3.5 = \mathbf{7}$$

Ejemplo 2: Variables Indicadoras (El Guardarropa)

Problema: n clientes dejan sus abrigos en un guardarropa. El asistente pierde los tickets y devuelve los n abrigos **totalmente al azar**. ¿Cuál es el número esperado de personas que reciben su abrigo correcto?

Sea X el total de personas con el abrigo correcto. Calcular la PMF exacta de X es un problema combinatorio complejo. En su lugar, usamos **Variables Indicadoras**:

$$X_i = \begin{cases} 1 & \text{si la persona } i \text{ recibe su abrigo correcto} \\ 0 & \text{si no} \end{cases}$$

Notemos que el total es $X = \sum_{i=1}^n X_i$. Por la **linealidad**:

$$E[X] = E\left[\sum_{i=1}^n X_i\right] = \sum_{i=1}^n E[X_i]$$

La probabilidad de que la persona i reciba su abrigo es simplemente $1/n$. Luego $E[X_i] = 1 \cdot (1/n) + 0 = 1/n$. Finalmente: $E[X] = n \cdot (1/n) = \mathbf{1}$. ¡Siempre es 1!

Desigualdad de Jensen

¿Qué pasa si la función $h(X)$ que aplicamos no es lineal (ej. un logaritmo o un cuadrado)? La esperanza **no** entra dentro de la función: $E[h(X)] \neq h(E[X])$.

Theorem (Desigualdad de Jensen)

Sea X una variable aleatoria integrable y φ una función **convexa** ($\cup$). Entonces:

$$\varphi(E[X]) \leq E[\varphi(X)]$$

Si la función es **cóncava** ($\cap$), como el $\log(x)$, la desigualdad se invierte:

$$\varphi(E[X]) \geq E[\varphi(X)]$$

Aplicación en IA: Como $\log(E[X]) \geq E[\log(X)]$, a menudo no podemos maximizar la verosimilitud directamente en variables latentes. Jensen nos permite construir un "Límite Inferior" (ELBO) y maximizar eso en su lugar (Base del algoritmo EM).

Varianza

Definición. La **varianza** de una variable aleatoria X con función de densidad f , denota por $Var(X)$ (a veces σ^2), está definida por

$$Var[X] = E[X - E[X]]^2 = E[X^2] - (E[X])^2$$

*Nota: La **Desviación Estándar** es simplemente $\sigma = \sqrt{Var(X)}$. Se utiliza porque tiene las mismas unidades físicas que la variable X .*

Esperanza y Varianza de Distribuciones Clásicas

Distribución	Notación	$E[X]$	$\text{Var}[X]$
Bernoulli	$\text{Bern}(p)$	p	$p(1 - p)$
Binomial	$\text{Bin}(n, p)$	np	$np(1 - p)$
Poisson	$\text{Pois}(\lambda)$	λ	λ
Uniforme	$\text{Unif}(a, b)$	$\frac{a+b}{2}$	$\frac{(b-a)^2}{12}$
Normal	$\mathcal{N}(\mu, \sigma^2)$	μ	σ^2
Exponencial	$\text{Exp}(\lambda)$	$1/\lambda$	$1/\lambda^2$
Gamma	$\text{Gamma}(\alpha, \beta)$	α/β	α/β^2
Student-t	t_ν	$0 \ (\nu > 1)$	$\frac{\nu}{\nu-2} \ (\nu > 2)$

Función Generadora de Momentos

Definición. La función generadora de momentos M de una variable aleatoria X está definida por

$$M(s) = E[\exp(sX)], \quad s \in \mathbb{R}.$$

Observación. Las funciones generadoras de momentos de dos variables aleatorias coinciden si y sólo si las variables aleatorias tienen la misma distribución de probabilidad.

Ejemplo de momentos, distribución Γ

Sea $X \sim \text{Gamma}(\alpha, \lambda)$. Para $s < \lambda$, la función generadora de momentos de X en s está dada por

$$\begin{aligned} M(s) &= E[\exp(sX)] = \int_0^\infty \exp(sx) \frac{\exp(-\lambda x) \lambda^\alpha x^{\alpha-1}}{\Gamma(\alpha)} \\ &= \left(\frac{\lambda}{\lambda - s} \right)^\alpha \int_0^\infty \frac{\exp(-(\lambda - s)x) (\lambda - s)^\alpha x^{\alpha-1}}{\Gamma(\alpha)} dx \\ &= \left(\frac{\lambda}{\lambda - s} \right)^\alpha. \end{aligned}$$

$$M'(s)|_{s=0} = \left(\left(\alpha \frac{\lambda^\alpha}{(\lambda - s)^{\alpha+1}} \right) \right) |_{s=0} = \frac{\alpha}{\lambda}$$

Variables Aleatorias Multivariadas

Distribuciones Conjuntas de Probabilidad

Definición. Sean $X_1, X_2, \dots, X_n$ variables aleatorias. Defina el vector $X = [X_1, \dots, X_n]^\top$ con distribución de probabilidad conjunta F definida por:

$$F(x_1, \dots, x_n) = P(X_1 \leq x_1, \dots, X_n \leq x_n)$$

Un vector aleatorio X que toma valores en $\mathbb{R}^n$ tiene una **función de densidad conjunta** f si:

$$P(X \in B) = \int_B f(x_1, \dots, x_n) dx_1 \dots dx_n$$

Distribuciones Marginales: Las funciones de densidad de cada componente se recuperan integrando sobre las demás variables. Para $X = [X_1, X_2]^\top$:

$$f_{X_1}(x_1) = \int_{-\infty}^{\infty} f(x_1, x_2) dx_2$$

Independencia de Variables Aleatorias

Ahora que tenemos la distribución conjunta, podemos definir la independencia formalmente.

Theorem (Caracterización de independencia)

*Las variables aleatorias $X_1, \dots, X_n$ con densidades marginales $f_{X_1}, \dots, f_{X_n}$ y densidad conjunta f son **independientes** si y sólo si su conjunta se factoriza:*

$$f(x_1, \dots, x_n) = f_{X_1}(x_1) \cdots f_{X_n}(x_n), \quad \text{para todo } x_1, \dots, x_n.$$

En Machine Learning, casi siempre asumimos que los datos de entrenamiento son **Independientes e Idénticamente Distribuidos (i.i.d.)**. Esto es lo que nos permitió usar la productoria $\prod$ en el Clasificador Naïve Bayes.

Covarianza y Correlación

Si dos variables *no* son independientes, necesitamos medir cómo varían juntas.

Covarianza: Mide la dirección de la dependencia lineal.

$$\text{Cov}(X, Y) = E[(X - E[X])(Y - E[Y])] = E[XY] - E[X]E[Y]$$

Correlación de Pearson: Es la covarianza estandarizada al rango $[-1, 1]$.

$$\rho_{X,Y} = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y}$$

Precaución: Si X e Y son independientes, entonces $\text{Cov}(X, Y) = 0$. Sin embargo, el inverso no siempre es cierto: covarianza cero sólo implica que no hay relación *lineal*, pero podría existir una fuerte dependencia no lineal.

La Gaussiana Multivariada

Es la generalización de la campana de Gauss a M dimensiones. Su forma está gobernada por el vector de medias μ y la **Matriz de Covarianza** Σ .

$$f(\mathbf{x}) = \frac{1}{(2\pi)^{M/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{x} - \mu)^\top \Sigma^{-1} (\mathbf{x} - \mu) \right)$$

Por qué es vital en IA:

PCA (Análisis de Componentes Principales): Busca los autovectores de la matriz Σ para reducir dimensiones.

Clustering (GMM): Los modelos de Mezcla Gaussiana asumen que el dataset está compuesto por varias "nubes" Gaussianas que debemos separar.

Anomalías: Los puntos con baja densidad $f(\mathbf{x})$ se clasifican como *outliers*.

El Puente: Probabilidad y Álgebra Lineal

¿De dónde sale la Matriz de Covarianza en la práctica?

Supongamos un *dataset* centrado en el origen representado por una matriz $\mathbf{X}$ de dimensiones $N \times M$ (N datos, M variables). La matriz de covarianza empírica se calcula matricialmente como:

$$\Sigma = \frac{1}{N-1} \mathbf{X}^\top \mathbf{X}$$

El truco de la SVD (Descomposición en Valores Singulares): Sabemos por Álgebra Lineal que podemos descomponer nuestros datos como $\mathbf{X} = \mathbf{U} \mathbf{S} \mathbf{V}^\top$.

$$\begin{aligned} \Sigma &= \frac{1}{N-1} (\mathbf{U} \mathbf{S} \mathbf{V}^\top)^\top (\mathbf{U} \mathbf{S} \mathbf{V}^\top) \\ &= \frac{1}{N-1} \mathbf{V} \mathbf{S} \underbrace{\mathbf{U}^\top \mathbf{U}}_I \mathbf{S} \mathbf{V}^\top \\ \Sigma &= \mathbf{V} \left(\frac{\mathbf{S}^2}{N-1} \right) \mathbf{V}^\top \end{aligned}$$

La Geometría de los Anillos: Vectores Singulares

La ecuación $\Sigma = \mathbf{V} \mathbf{D} \mathbf{V}^\top$ (donde $\mathbf{D} = \frac{\mathbf{S}^2}{N-1}$) es literalmente la descomposición espectral de la covarianza.

Esto le da un significado físico directo a la SVD sobre nuestros "anillos" de probabilidad Gaussiana:

La orientación (Vectores Singulares $\mathbf{V}$): Las columnas de $\mathbf{V}$ son vectores ortogonales que nos dicen **hacia dónde están rotados** los anillos. Son los ejes principales de la elipse.

El estiramiento (Valores Singulares $\mathbf{S}$): Los valores singulares σ_i (al cuadrado y normalizados) nos dicen **cuánto miden los semiejes** del anillo. Un valor singular grande significa que el anillo está muy estirado en esa dirección (muchas varianzas).

Aplicación IA: Reducción de Dimensionalidad (PCA)

Entender que los anillos de probabilidad están definidos por la SVD es el núcleo del **Análisis de Componentes Principales (PCA)**.

El Problema (Maldición de la Dimensionalidad): Si tenemos un dataset con 1000 dimensiones, pero calculamos su SVD y vemos que solo 5 valores singulares son grandes y los otros 995 son casi cero...

La Solución Geométrica: Significa que nuestro "anillo" de 1000 dimensiones es en realidad un panqueque plano y aplastado que vive en un subespacio de 5 dimensiones. Podemos proyectar todos nuestros datos sobre los 5 vectores singulares principales (V) sin perder casi nada de información de probabilidad, acelerando brutalmente nuestros modelos de Machine Learning.

Aplicación: Algoritmos Aleatorizados

Probabilidad en el Diseño de Software

Hasta ahora hemos usado la probabilidad para modelar datos. Pero también podemos inyectar aleatoriedad *dentro* de nuestros propios algoritmos para hacerlos más rápidos y robustos frente a los peores escenarios.

Algoritmos "Las Vegas": Siempre entregan la respuesta correcta, pero su tiempo de ejecución es una **variable aleatoria**. Nuestro objetivo como Científicos de Datos es demostrar que el **Valor Esperado** de ese tiempo de ejecución es muy bajo.

Ejemplo clásico: Quicksort y Búsqueda de la Mediana.

Por qué usar aleatoriedad: Evita que un "hacker" o un *dataset* maliciosamente ordenado fuerce a nuestro algoritmo a caer en su peor caso ($O(N^2)$).

Problema de ordenar una lista

El problema: Dada una lista de números S de tamaño n queremos un algoritmo que ordene esta lista.

En principio, podemos pensar en un algoritmo que compare cada número de la lista con el resto, obteniendo $n - 1$ comparaciones para el primer elemento, luego $n - 1$ comparaciones para el segundo, hasta 2 comparaciones para el elemento $n - 2$ y 1 comparación para el elemento $n - 1$. Así obtenemos un algoritmo de un total de

$$\frac{n(n-1)}{2} = \sum_{i=1}^n (n-i) \text{ comparaciones}$$

es decir $\mathcal{O}(n^2)$ operaciones.

Algoritmo Aleatorio

Algorithm 1: Quicksort

Input : Una lista $S = \{x_1, \dots, x_n\}$ de n elementos distintos.

Output: Elementos de lista S ordenados.

if S no tiene elementos o sólo un elemento **then**

 | return S

Escoger un elemento x de S ;

Comparar cada uno de los elementos de S con x y crear 3 sublistas;

 1. S_1 tiene todos los elementos S menores a x ;

 2. S_2 tiene todos los elementos S mayores a x ;

Ordenar S_1 usando Quicksort;

Ordenar S_2 usando Quicksort;

return $\{S_1, x, S_2\}$

Quicksort Aleatorizado

El problema: Queremos ordenar un arreglo de n elementos. El Quicksort determinista elige siempre el primer elemento como "pivote". Si el arreglo ya está ordenado al revés, el tiempo de ejecución colapsa a $O(n^2)$.

La Solución Aleatorizada: Elegimos el pivote **uniformemente al azar** entre los elementos disponibles.

Para calcular el tiempo de ejecución esperado, nos interesa el número total de comparaciones C . Usamos el mismo truco que en el problema del Guardarropa (**Variables Indicadoras**):

$$X_{ij} = \begin{cases} 1 & \text{si el elemento } i \text{ es comparado con el } j \\ 0 & \text{si no son comparados} \end{cases}$$

El total de comparaciones es la suma de todas las posibles variables indicadoras:

$$C = \sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}$$

El Poder de la Linealidad de la Esperanza

Aplicamos la linealidad de la esperanza:

$$E[C] = E \left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij} \right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}]$$

Sabemos que $E[X_{ij}] = P(X_{ij} = 1)$, que es la probabilidad de que i y j sean comparados. Esto solo ocurre si i o j es elegido como pivote *antes* que cualquier otro elemento entre ellos. Como hay $j - i + 1$ elementos en ese rango:

$$P(X_{ij} = 1) = \frac{2}{j - i + 1}$$

Resolviendo la doble sumatoria, obtenemos el tiempo esperado universal:

$$E[C] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j - i + 1} \approx 2n \ln(n) = O(n \log n)$$

Conclusión: Con un simple número aleatorio, garantizamos un rendimiento óptimo en promedio, sin importar cómo vengan los datos.

Tiempo de cómputo esperado para Quicksort

Theorem

*Suponga que cuando un pivot es escogido por el algoritmo **Quicksort** aleatorio, este es escogido independientemente y uniformemente dentro de todas las posibilidades. Entonces, para todo input, el número esperado de comparaciones realizadas por el algoritmo es de $2n \log(n) + \mathcal{O}(n)$.*

Algoritmo aleatorio para el cómputo de la mediana

Dado un conjunto S de n elementos. La mediana de S es un elemento m de S tal que al menos $\lfloor \frac{n}{2} \rfloor$ elementos de S son menores o iguales que m . Si los elementos de S son distintos entonces m es el $(\lceil \frac{n}{2} \rceil)$ -ésimo elemento de S .

Algoritmo de la Mediana (Quickselect)

El problema: Encontrar la mediana (o el k -ésimo elemento más pequeño) en un *dataset* desordenado de n elementos, *sin tener que ordenar todo el arreglo*.

El algoritmo:

Elegimos un pivote al azar.

Partimos el arreglo en mayores y menores (como en Quicksort).

Descartamos la mitad donde sabemos que no está la mediana, y repetimos solo en la mitad relevante.

Análisis del Valor Esperado: En promedio, el pivote aleatorio descartará la mitad de los elementos en cada paso. El número esperado de operaciones es la serie geométrica:

$$E[\text{Tiempo}] = n + \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \cdots \approx 2n = O(n)$$

Algoritmo Mediana aleatoria

Algorithm 2: Mediana Aleatoria

Input : Una lista $S = \{x_1, \dots, x_n\}$ de n elementos.

Output: La mediana m de S .

Escoger un conjunto R con $\lceil n^{3/4} \rceil$ elementos en S elegidos independientemente, uniformemente y aleatoriamente con reemplazo ;

Ordenar el conjunto R ;

Sea d el $(\lfloor n^{3/4}/2 - \sqrt{n} \rfloor)$ -ésimo elemento mas pequeño de R ordenado;

Sea u el $(\lceil n^{3/4}/2 - \sqrt{n} \rceil)$ -ésimo elemento mas pequeño de R ordenado;

Comparar cada elemento de S con d y u ;

Computar

$C = \{x \in S : d \leq x \leq u\};$

$l_d = \{x \in S : x < d\};$

$l_u = \{x \in S : x > u\};$

end

Algoritmo Mediana aleatoria

Algorithm 3: Mediana Aleatoria

Input : Una lista $S = \{x_1, \dots, x_n\}$ de n elementos.

Output: La mediana m de S .

if $l_d > n/2$ o $l_u > n/2$ **then**

 | Fail

if $|C| \leq 4n^{3/4}$ **then**

 | Ordena C

else

 | Fail

end

return $(\lfloor n/2 \rfloor - l_d + 1)$ -ésimo elemento de C ordenado.

Algoritmo Mediana Aleatoria

Theorem

La probabilidad de que falle el algoritmo Mediana Aleatoria está acotada por $n^{-1/4}$.

La Maldición de la Dimensionalidad

La Maldición de la Dimensionalidad

Un fenómeno que complica el trabajo de un Científico de Datos en dimensiones altas (datasets con cientos de *features*) es que **el espacio crece exponencialmente más rápido que los datos**.

Consecuencias en Machine Learning:

Sparcity (Escasez): Para mantener la misma densidad de información que en 1D, necesitamos un tamaño de muestra que crece exponencialmente con la dimensión.

Colapso de Distancias: En altas dimensiones, la distancia entre un punto y su "vecino más cercano" se vuelve casi igual a la distancia con su "vecino más lejano".

Fallo algorítmico: Modelos basados en distancias (como k -NN o clustering clásico) fallan miserablemente. ¡Por eso necesitamos PCA o Redes Neuronales!

Ilustración Matemática: La Esfera vs. El Hipercubo

Considere un vector aleatorio $X = (X_1, \dots, X_n)$ donde cada X_i es Uniforme independiente en $[-1, 1]$. Todo nuestro *dataset* vive en un hipercubo de volumen 2^n .

¿Cuál es la probabilidad de que un punto al azar caiga cerca del centro, es decir, dentro de la esfera unitaria $B_n = \{X : \sum X_i^2 \leq 1\}$?

$$P(X \in B_n) = \frac{\text{Vol}(B_n)}{\text{Vol}(\text{Cubo})} = \frac{\pi^{n/2}}{2^n \Gamma(n/2 + 1)}$$

Dimensión (n)	2D	3D	4D	10D	20D
Probabilidad $\in B_n$	78.5%	52.4%	30.8%	0.24%	0.000002%

Conclusión anti-intuitiva: En altas dimensiones, el "centro" del espacio está completamente vacío. ¡El 99.99% de los datos viven escondidos en las esquinas del hipercubo!

Desigualdades de Concentración

El Riesgo Empírico vs. Riesgo Real

El Problema Fundamental del Machine Learning: Entrenamos nuestros modelos usando una muestra finita de n datos. Calculamos el **Error Empírico** ($\bar{X}_n$) en ese dataset. Sin embargo, lo que realmente nos importa es el **Error Real** (μ) sobre datos futuros invisibles.

¿Cómo podemos garantizar matemáticamente que $\bar{X}_n \approx \mu$?

Las **Desigualdades de Concentración** nos dan límites matemáticos duros (garantías) de que el promedio de una muestra no se desviará demasiado de su valor esperado, sin importar qué distribución extraña tengan los datos.

Desigualdad de Markov (El límite más básico)

Theorem (Markov)

Sea X una variable aleatoria que solo toma valores no negativos ($X \geq 0$). Entonces, para toda constante $a > 0$:

$$P(X \geq a) \leq \frac{E[X]}{a}$$

Ejemplo en IA (Tasa de Error): Supongamos que el error esperado de nuestro modelo en producción es $E[\text{Error}] = 5\%$. ¿Cuál es la probabilidad del peor escenario, es decir, que el error sea mayor al 20% en un lote de datos?

$$P(\text{Error} \geq 20\%) \leq \frac{5\%}{20\%} = 0.25 \quad (25\%)$$

Desigualdad de Markov

Demostración. Como $X > 0$, tenemos lo siguiente

$$\begin{aligned} E[X] &= \int_0^{\infty} x f_x(x) dx \\ &= \int_0^t x f_x(x) dx + \int_t^{\infty} x f_x(x) dx \\ &\geq \int_t^{\infty} x f_x(x) dx \\ &\geq t \int_t^{\infty} f_x(x) dx \\ &= P(X \geq t). \end{aligned}$$

Ejemplo: desigualdad de Markov

Probabilidad de obtener al menos $3n/4$ caras en una sucesión de n lanzamientos de monedas

$$X_i = \begin{cases} 1 & \text{si la } i\text{-ésima moneda es cara,} \\ 0 & \text{en otro caso,} \end{cases} \quad X = \sum_{i=1}^n X_i, \quad \text{número de caras.}$$

Entonces,

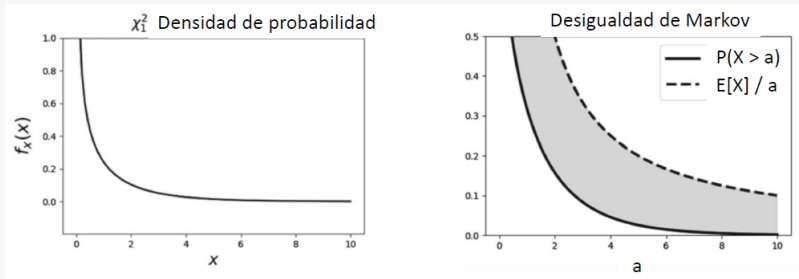
$$E[X_i] = P(X_i = 1) = \frac{1}{2}, \quad E[X] = \sum_{i=1}^n E[X_i] = \frac{n}{2}.$$

Aplicando la desigualdad de Markov

$$P(X \geq \frac{3n}{4}) \leq \frac{E[X]}{3n/4} = \frac{n/2}{3n/4} = \frac{2}{3}.$$

Desigualdad de Markov

Observación. Como vemos en la demostración la desigualdad de Markov omite una parte de la integral, es decir, hay una diferencia (considerable) entre ambos lados de la desigualdad.



Desigualdad de Markov

Observación. Un caso especial de la desigualdad de Markov se da cuando $X = |Y - \mu|^2$, para alguna variable aleatoria Y con media μ y varianza σ^2 . Al aplicar la desigualdad de Markov, nos queda

$$P(|Y - \mu| \geq a) = P(|Y - \mu|^2 \geq a^2) \leq \frac{E[|Y - \mu|^2]}{a^2} = \frac{\sigma^2}{a^2}$$

Esta desigualdad se conoce como la **desigualdad de Chebyshev**.

Desigualdad de Chebyshev

Theorem (Chebyshev)

Si conocemos la varianza ($\text{Var}[X] = \sigma^2$), podemos acotar qué tan probable es que la variable se aleje de su propia media μ :

$$P(|X - \mu| \geq a) \leq \frac{\sigma^2}{a^2}$$

Aplicación: Reemplazando $a = k\sigma$, obtenemos $P(|X - \mu| \geq k\sigma) \leq \frac{1}{k^2}$.

Esto demuestra que, sin importar cuál sea la distribución subyacente de los datos (normal, asimétrica, bimodal), la probabilidad de que un dato caiga a más de 3 desviaciones estándar de la media es siempre menor a $1/9 \approx 11\%$.

Ejemplo: desigualdad de Chebyshev

Lanzamiento de una moneda n veces. Queremos acotar la probabilidad de obtener mas que $3n/4$ caras. Nuevamente hacemos

$$X_i = \begin{cases} 1 & \text{si el } i\text{-ésimo lanzamiento es cara,} \\ 0 & \text{en otro caso,} \end{cases} \quad X = \sum_{i=1}^n X_i, \quad \text{número de caras.}$$

Entonces

$$E[X_i] = \frac{1}{2}, \quad E[X_i^2] = \frac{1}{4}, \quad \text{Var}[X_i] = \frac{1}{4}, \quad \text{Var}[X] = \frac{n}{4}.$$

Aplicamos la desigualdad de Chebyshev

$$P(X \geq \frac{3}{4}n) \leq P(|X - E[X]| \geq \frac{n}{4}) \leq \frac{\text{Var}[X]}{(n/4)^2} = \frac{4}{n}.$$

Ejemplo: coleccionador de cupones

Sea X una variable aleatoria que representa el número de cajas para coleccionar los n cupones tiene esperanza

$$E[X] = nH_n$$

Así, al aplicar la desigualdad de Markov obtenemos

$$P(X \geq 2nH_n) \leq \frac{1}{2}$$

Definimos

X_i : variables aleatorias geométricas con parámetro $\frac{(n-i+1)}{n}$

Entonces

$$X = \sum_{i=1}^n X_i, \quad X_i \text{ independientes}, \quad Var[X] = \sum_{i=1}^n Var[X_i]$$

Ejemplo: coleccionador de cupones

$$\text{Var}[X_i] = \frac{1 - (n - i + 1)/n}{((n - i + 1)/n)^2} \leq \frac{n^2}{(n - i + 1)^2}$$

Se sigue que

$$\text{Var}[X] \leq \sum_{i=1}^n \frac{n^2}{(n - i + 1)^2} \leq \frac{n^2 \pi^2}{6}$$

Entonces La desigualdad de Chebyshev implica

$$P(|X - nH_n| \geq nH_n) \leq \frac{n^2 \pi^2 / 6}{n^2 H_n^2} \approx \mathcal{O}(1/(\log(n))^2)$$

Ejemplo: coleccionador de cupones

$$\text{Var}[X_i] = \frac{1 - (n - i + 1)/n}{((n - i + 1)/n)^2} \leq \frac{n^2}{(n - i + 1)^2}$$

Se sigue que

$$\text{Var}[X] \leq \sum_{i=1}^n \frac{n^2}{(n - i + 1)^2} \leq \frac{n^2 \pi^2}{6}$$

Entonces La desigualdad de Chebyshev implica

$$P(|X - nH_n| \geq nH_n) \leq \frac{n^2 \pi^2 / 6}{n^2 H_n^2} \approx \mathcal{O}(1/(\log(n))^2)$$

$$P(X \geq 2nH_n) \leq \frac{n^2 \pi^2 / 6}{n^2 H_n^2} \approx \mathcal{O}(1/(\log(n))^2)$$

Ejemplo: lanzamiento de dado

Problema. Suponga que se lanzan dos dados n veces. Es posible dar una buena estimación del total valor de los n lanzamientos?

Solución. Si calculamos el valor esperado y la varianza, encontramos que

$$\mu = E[X] = 7n, \quad \sigma^2 = \frac{35}{6}n$$

Al aplicar la desigualdad de Chebyshev, encontramos que la suma final es en el intervalo

$$\left[7n - 10\sqrt{\frac{35n}{6}}, 7n + 10\sqrt{\frac{35n}{6}}\right]$$

en al menos el 99% de todos los experimentos.

Desigualdad de Hoeffding

Theorem (Hoeffding)

Sean $X_1, X_2, \dots, X_n$ variables i.i.d. acotadas ($a \leq X_i \leq b$). Sea $\bar{X}_n$ el promedio de la muestra y μ la media real. Para cualquier $\varepsilon > 0$:

$$P(|\bar{X}_n - \mu| \geq \varepsilon) \leq 2 \exp \left(-\frac{2n\varepsilon^2}{(b-a)^2} \right)$$

A diferencia de Markov o Chebyshev (que decaen polinomialmente), Hoeffding garantiza que la probabilidad de error decrece **exponencialmente** a medida que aumentamos el tamaño de la muestra n . Esta es la base matemática de la teoría PAC (*Probably Approximately Correct*) en Machine Learning.

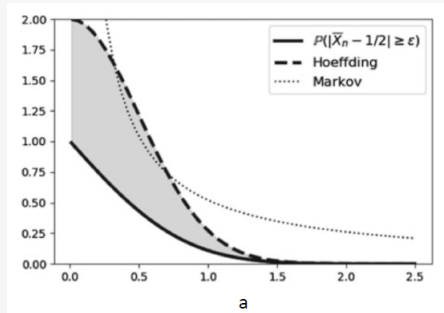
Desigualdad de Hoeffding

Reescribimos:

$$P(|\bar{X}_n - \mu| \geq a) \leq 2 \exp \left(-\frac{2n^2 a^2}{(b-a)^2} \right)$$

Si $X_i \sim \mathcal{U}[0, 1]$, $a \leq X_i \leq b$

$$P(|\bar{X}_n - \mu| \geq a) \leq \frac{\sigma^2}{na^2}$$



Ejemplo Práctico: ¿Cuántos datos necesito etiquetar?

Problema: Quieres garantizar con un **95% de confianza** que el error que ves en tu *dataset* de entrenamiento está, a lo sumo, a $\varepsilon = 0.05$ (5%) del error real del modelo en producción. ¿De qué tamaño (n) debe ser tu dataset?

Solución con Hoeffding: Queremos que la probabilidad de que se rompa nuestra tolerancia sea ≤ 0.05 (para tener un 95% de certeza).

$$\begin{aligned}2 \exp(-2n\varepsilon^2) &\leq \delta \\2 \exp(-2n(0.05)^2) &\leq 0.05 \\-0.005n &\leq \ln(0.025) \approx -3.689 \\n &\geq 738\end{aligned}$$

Conclusión: Solo necesitas etiquetar **738 imágenes** para tener garantías estadísticas sólidas, sin importar si tienes 1 millón de imágenes en total.

Muestras

¿Qué es una Muestra (Sample)?

En la práctica, no tenemos acceso a toda la población (la distribución real $f(x)$). Solo tenemos un *dataset*.

Definición Formal: Una **muestra** de tamaño n es una colección de variables aleatorias $X_1, X_2, \dots, X_n$ que son independientes e idénticamente distribuidas (i.i.d.) provenientes de la población.

A partir de la muestra, calculamos **estimadores** (que también son variables aleatorias):

Estimadores Fundamentales

Media Muestral: $\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i \implies E[\bar{X}] = \mu, \quad \text{Var}[\bar{X}] = \frac{\sigma^2}{n}$

Varianza Muestral: $S^2 = \frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2 \implies E[S^2] = \sigma^2$

Ejemplo Práctico: Testeo Agrupado (Pooled Testing)

El Problema: Necesitamos testear a 1000 conscriptos del ejército para detectar una enfermedad rara que afecta solo al 0.2% de la población ($p = 0.002$). Hacer 1000 tests individuales es muy costoso.

La Solución Algorítmica:

Dividimos a los 1000 conscriptos en 10 grupos de 100 personas.

Mezclamos una fracción de la sangre de las 100 personas de un grupo y hacemos **un solo test** a la mezcla.

Si la mezcla da negativo, las 100 personas están sanas (usamos 1 test).

Si la mezcla da positivo, hacemos 100 tests individuales adicionales a ese grupo (usamos $1 + 100 = 101$ tests).

Preguntas: a) ¿Cuál es el máximo número de tests posibles?

b) ¿Cuál es el mínimo número de tests posibles?

Ejemplo Práctico: Testeo Agrupado (Pooled Testing)

El Problema: Necesitamos testear a 1000 conscriptos del ejército para detectar una enfermedad rara que afecta solo al 0.2% de la población ($p = 0.002$). Hacer 1000 tests individuales es muy costoso.

La Solución Algorítmica:

Dividimos a los 1000 conscriptos en 10 grupos de 100 personas.

Mezclamos una fracción de la sangre de las 100 personas de un grupo y hacemos **un solo test** a la mezcla.

Si la mezcla da negativo, las 100 personas están sanas (usamos 1 test).

Si la mezcla da positivo, hacemos 100 tests individuales adicionales a ese grupo (usamos $1 + 100 = 101$ tests).

Preguntas: a) ¿Cuál es el máximo número de tests posibles? **1010 tests.**

b) ¿Cuál es el mínimo número de tests posibles? **10 tests.**

Optimización de Recursos usando Valor Esperado

c) ¿Cuántos tests **esperamos** realizar en promedio con este método?

Sea Z el número total de tests. Definimos la variable indicadora para cada grupo $i \in \{1, \dots, 10\}$:

$$Z_i = \begin{cases} 1 & \text{si el test de la mezcla del grupo } i \text{ da positivo} \\ 0 & \text{si da negativo} \end{cases}$$

El total de tests es $Z = 10 + 100 \sum_{i=1}^{10} Z_i$. Aplicando la linealidad de la esperanza:

$$E[Z] = 10 + 100 \sum_{i=1}^{10} E[Z_i]$$

¿Cuál es la probabilidad de que un grupo dé positivo? Es el complemento de que *todos* estén sanos:

$$E[Z_i] = P(Z_i = 1) = 1 - P(100 \text{ personas sanas}) = 1 - (1 - 0.002)^{100} \approx \mathbf{0.181}$$

Leyes Límite (LLN y CLT)

El Comportamiento Asintótico de la Muestra

En las secciones anteriores vimos que el promedio muestral $\bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i$ es una variable aleatoria con $E[\bar{X}_n] = \mu$ y varianza $\text{Var}[\bar{X}_n] = \frac{\sigma^2}{n}$.

La Pregunta Asintótica: ¿Qué sucede matemáticamente con esta variable aleatoria a medida que recolectamos "Big Data" ($n \rightarrow \infty$)?

Dos teoremas fundamentales responden a esta pregunta:

Ley de los Grandes Números (LLN): Nos dice *hacia dónde* converge el promedio.

Teorema del Límite Central (CLT): Nos dice *cómo* se distribuyen los errores alrededor de ese promedio mientras converge.

Ley de los Grandes Números (LLN)

Theorem (Ley Débil de los Grandes Números)

Sean $X_1, X_2, \dots, X_n$ variables aleatorias i.i.d. con media μ y varianza finita. Si definimos la media muestral como $\bar{X}_n = \frac{1}{n} \sum X_i$, entonces para cualquier $\varepsilon > 0$:

$$\lim_{n \rightarrow \infty} P(|\bar{X}_n - \mu| \geq \varepsilon) = 0$$

Significado Práctico: Si recolectamos suficientes datos ($n \rightarrow \infty$), el promedio de nuestra muestra (*dataset*) convergerá inexorablemente al valor real de la población. Esto justifica por qué recolectar "Big Data" mejora el entrenamiento empírico de nuestros algoritmos.

Ley Débil de los Grandes Números (LLN)

Theorem (Ley Débil)

Sean $X_1, X_2, \dots, X_n$ variables aleatorias i.i.d. con media μ y varianza σ^2 finita. El promedio muestral $\bar{X}_n$ converge a μ **en probabilidad**.

$$\lim_{n \rightarrow \infty} P(|\bar{X}_n - \mu| \geq \varepsilon) = 0 \quad \text{para cualquier } \varepsilon > 0$$

La Demostración (El poder de Chebyshev): ¡Con lo que aprendimos hace 10 minutos, podemos demostrar este teorema fundamental en una sola línea!

$$P(|\bar{X}_n - \mu| \geq \varepsilon) \leq \frac{\text{Var}[\bar{X}_n]}{\varepsilon^2} = \frac{\sigma^2}{n\varepsilon^2}$$

Es evidente que cuando $n \rightarrow \infty$, el término derecho colapsa a 0.

Ley débil de grandes números

Considere una sucesión de ensayos de Bernoulli con

$$X_i = \begin{cases} 1 & \text{si el } i\text{-ésimo ensayo es exitoso,} \\ 0 & \text{en otro caso.} \end{cases}$$

Entonces

$$\bar{X}_n = \sum_{i=1}^n \frac{X_i}{n}$$

es la frecuencia relativa de ensayos exitosos en n ensayos y $E[X_i]$ es la probabilidad p de éxito del ensayo i -ésimo. La Ley débil nos dice que si consideramos a número n fijo suficientemente grande, la probabilidad que la frecuencia relativa converja a p está garantizada.

Ley Fuerte de los Grandes Números (SLLN)

Theorem

Sean $X_1, X_2, \dots$ variables aleatorias i.i.d. con $E[|X_i|] < \infty$ y media μ . Entonces, para todo $\varepsilon > 0$:

$$\lim_{n \rightarrow \infty} P \left(\sup_{k \geq n} |\bar{X}_k - \mu| > \varepsilon \right) = 0$$

En otras palabras, $\bar{X}_n$ converge a μ a.s. (casi seguramente).

Ley Fuerte de los Grandes Números (SLLN)

La Ley Débil nos asegura que una "foto" en el futuro lejano se verá bien. La **Ley Fuerte** es mucho más estricta: nos garantiza que la "película" completa del promedio se estabilizará sin saltos anómalos.

Theorem (Ley Fuerte)

*Bajo condiciones similares (solo requiere $E[|X_i|] < \infty$), el promedio muestral converge a la media real **casi seguramente** (c.s.):*

$$P\left(\lim_{n \rightarrow \infty} \bar{X}_n = \mu\right) = 1$$

Significado en Inteligencia Artificial (ERM)

En Machine Learning nunca podemos calcular el "Riesgo Real" (el error sobre todos los datos posibles del universo). Solo podemos calcular el "Riesgo Em-

Diferencia Clave: Ley Débil vs. Ley Fuerte

¿Por qué tenemos dos leyes si ambas concluyen que $\bar{X}_n$ se acerca a μ ? La gran diferencia radica en los **tipos de convergencia** a lo largo del tiempo.

Ley Débil (La "Foto")

Convergencia en Probabilidad

Si tomas una foto del error en un paso n inmenso, es casi seguro que estará muy cerca de μ .

Sin embargo, permite que, si sigues observando hasta el infinito, la línea dé "saltos anómalos" de vez en cuando.

Uso teórico: Sirve para demostrar que un estimador estadístico es *consistente*.

Ley Fuerte (La "Película")

Convergencia Casi Segura

Si observas la *trayectoria completa* desde hoy hasta el infinito, se anclará a μ y nunca más divergirá.

Las trayectorias que dan "saltos locos" infinitamente tienen una probabilidad estricta de 0.

Uso práctico (IA): Garantiza que el entrenamiento iterativo de tu modelo será estable en el tiempo.

Más allá de Chebyshev: Garantías en el Tiempo

La desigualdad de Chebyshev es una herramienta de "punto fijo": nos dice qué tan probable es que $\bar{X}_n$ esté lejos de μ en un **paso n específico**.

Sin embargo, en procesos iterativos (como el entrenamiento de una red neuronal), nos interesa saber si el proceso se mantuvo "bajo control" en **todo momento**.

Theorem (Desigualdad de Kolmogorov)

Sean $X_1, \dots, X_n$ variables aleatorias independientes con $E[X_i] = 0$ y $\text{Var}(X_i) = \sigma_i^2 < \infty$. Sea $S_k = \sum_{i=1}^k X_i$ la suma parcial. Entonces, para cualquier $\varepsilon > 0$:

$$P\left(\max_{1 \leq k \leq n} |S_k| \geq \varepsilon\right) \leq \frac{\text{Var}(S_n)}{\varepsilon^2} = \frac{\sum_{i=1}^n \sigma_i^2}{\varepsilon^2}$$

Chebyshev vs. Kolmogorov: ¿Cuál es la diferencia?

¿Por qué es tan sorprendente este resultado?

Chebyshev: Acota la probabilidad de que el **último punto** de la trayectoria sea extremo.

Kolmogorov: Acota la probabilidad de que **cualquier punto** de la trayectoria (desde el paso 1 hasta el n) haya superado el umbral.

Conexión con la Ley Fuerte (SLLN)

Esta es la pieza que faltaba: al acotar el **máximo** de las sumas parciales, Kolmogorov nos permite usar el Lema de Borel-Cantelli para demostrar que las trayectorias "malas" (que se escapan de la media) dejan de ocurrir casi seguramente. Es el motor matemático que permite pasar de la Ley Débil a la Ley Fuerte.

Teorema del Límite Central (CLT)

La LLN nos dice que el promedio converge a la media, pero *¿cómo* es la distribución de ese promedio mientras converge?

Theorem (Teorema del Límite Central)

La variable estandarizada del promedio muestral converge en distribución a una Normal Estándar $\mathcal{N}(0, 1)$:

$$\lim_{n \rightarrow \infty} P\left(\frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} \leq t\right) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^t \exp\left(-\frac{x^2}{2}\right) dx$$

El "Milagro" Estadístico: No importa si los datos originales son uniformes, exponenciales o bimodales. Si promediamos suficientes muestras independientes, la distribución de ese promedio **siempre** será una campana de Gauss.

De la Convergencia a la Incertidumbre (La Transición)

La **Ley Fuerte (SLLN)** nos dice hacia dónde vamos: sabemos que el promedio muestral $\bar{X}_n$ colapsa exactamente al valor real μ .

Pero la SLLN no nos dice nada sobre **cómo es la forma** de ese error mientras estamos convergiendo. Si amplificamos el error multiplicándolo por $\sqrt{n}$ para evitar que colapse a cero, descubrimos el patrón más asombroso de la naturaleza.

Teorema del Límite Central (CLT)

Theorem (Teorema del Límite Central)

Sean $X_1, X_2, \dots, X_n$ variables aleatorias i.i.d. con media μ y varianza $0 < \sigma^2 < \infty$. La variable aleatoria estandarizada de la suma (o del promedio) converge **en distribución** a una Normal Estándar $\mathcal{N}(0, 1)$ cuando $n \rightarrow \infty$:

$$Z_n = \frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} \xrightarrow{d} \mathcal{N}(0, 1)$$

Equivalentemente:

$$\lim_{n \rightarrow \infty} P(Z_n \leq t) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^t e^{-x^2/2} dx$$

El "Milagro" Estadístico en Ciencia de Datos

Por qué es matemáticamente mágico: El teorema **NO** exige que conozcamos la distribución original de los datos. La población original X_i puede ser asimétrica, bimodal, o una caja negra incomprensible. Al promediar suficientes muestras, el resultado **siempre** será una campana de Gauss perfecta.

Justificación en Inteligencia Artificial

En un *dataset* real, el "ruido" o error de una medición es la suma de miles de pequeños factores independientes (temperatura del sensor, estática, errores humanos, etc.). Por el CLT, la suma de estos miles de errores inobservables **debe** tener una distribución Normal. ¡Por esto las Redes Neuronales usan el **Error Cuadrático Medio (MSE)** asumiendo errores Gaussianos!

Teorema del Límite Central Multivariado

Como en Inteligencia Artificial trabajamos con vectores de múltiples características (*features*), el CLT se generaliza naturalmente al espacio M -dimensional.

Theorem (CLT Multivariado)

Sean $\mathbf{X}_1, \mathbf{X}_2, \dots, \mathbf{X}_n$ vectores aleatorios i.i.d. con vector de medias $\boldsymbol{\mu}$ y matriz de covarianza finita $\boldsymbol{\Sigma}$. Defina el vector promedio $\bar{\mathbf{X}}_n$. Entonces:

$$\sqrt{n}(\bar{\mathbf{X}}_n - \boldsymbol{\mu}) \xrightarrow{d} \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}) \quad \text{cuando } n \rightarrow \infty$$

Esta es la razón teórica por la cual en la Sección 3 modelamos los clusters de datos (GMM, PCA) usando los "anillos" de la matriz de covarianza de la Gaussiana Multivariada. La geometría de la IA es Gaussiana por culpa del CLT.

Garantías vs. Asintóticas

Retomemos la pregunta del Data Scientist: queremos estimar μ con un error menor a $\varepsilon = 1$ y un **99% de confianza**, sabiendo que $\sigma^2 = 4$. ¿Cuántos datos (n) necesitamos?

Vía Chebyshev (Garantía Dura)

$$P(|\bar{X} - \mu| \geq 1) \leq \frac{4}{n(1)^2} \leq 0.01$$

$$\implies n \geq 400$$

Es una verdad absoluta.

Se prepara para el peor escenario del universo.

Vía CLT (Aproximación)

Buscamos $P(|Z| \leq \sqrt{n}/2) = 0.99$.

Sabemos que $Z_{0.995} \approx 2.576$.

$$\implies n \geq 27$$

Asume que para $n = 27$ ya somos una campana de Gauss.

Mucho más barato computacionalmente.



PONTIFICIA
UNIVERSIDAD
CATÓLICA
DE CHILE